

Decomposition-Based Generation of Executable CAD Programs from Natural Language

Aayush Iyengar
Purdue University
iyengar6@purdue

Abstract

This paper contains a holistic review of three recently published articles on the relationship between natural language processing with program synthesis and large-language-model (LLM) tool use. The analysis of the contents of three articles, Toolformer (Schick et al., 2023), DECINT (Jhamtani et al., 2023), and Gorilla (Patil et al., 2024) was leveraged into a novel system that combines their core ideas into a much less explored domain: natural-language driven CAD geometry creation. The system detailed in this paper, **CADFormer**, maps complex user input strings through a hierarchical decomposition stage derived from DECINT, implements a methodical plan to increase command complexity from the initial decomposition, and uses a Gorilla-inspired API code generation stage, culminating in a CadQuery Python program that creates 3D geometry. The effectiveness of CADFormer was tested over 36 CAD prompts across three difficulty levels, covering topics from simple operations to parametric assemblies, where the execution success rate was measured. CADFormer achieved an overall execution success rate of 86.1%, outperforming direct generation (77.8%), chain-of-thought (69.4%), and decomposition without retrieval (52.8%), demonstrating that retrieval-augmented, interleaved decomposition is a viable approach for NL-to-CAD program synthesis.

1 Introduction

Large language models (LLMs) currently have progressed significantly in their ability to put together executable programs from natural language. Typical LLM-based program synthesis tasks involve creating general-executable code, not code that deals with a geometric modeling kernel. Generating programs for such systems requires an understanding of the relationship between parts in 2D and 3D space, design constraints, and an understanding of

how sequences of parametric modeling operations affect the final geometry.

CAD (Computer-Aided Design) is a great test for NL-to-program systems because it requires more than just creating runnable code. An input like creating a hollow beam with a certain wall thickness and bold pattern must be split into several smaller modeling choices, such as creating the solid shapes, removing material, and placing repeated features in the requested regions. To be successful in generating such a model, the system must have a good grasp of geometry and how dimensions from prior steps will affect future steps. When creating CAD, mistakes do not simply cause runtime errors; rather, the code may complete fully while generating an incorrect shape.

This paper makes two main contributions. It provides an in-depth review of three important papers on LLM tools and API use: Toolformer (Schick et al., 2023), DECINT (Jhamtani et al., 2023), and Gorilla (Patil et al., 2024), focusing on their methods, experiments, and limitations. Then, it presents CADFormer, a system that applies ideas from these papers to the CAD domain. CADFormer sequentially decomposes an input into sub-tasks as demonstrated in DECINT, uses Gorilla-inspired retrieval-augmented code generation, and a CadQuery backend to turn natural-language design requests into executable CAD programs. The paper then describes the implementation process, evaluates the system’s performance against several inputs, and compares the results to the ideas from the three reviewed papers.

2 Critical Review of Selected Papers

2.1 Toolformer: Language Models Can Teach Themselves to Use Tools

Introduction and Motivation. Toolformer (Schick et al., 2023) addresses one of the fundamental issues in LLM design today, which is

that even though larger models generalize well across many tasks, they sometimes struggle with problems that require exact arithmetic, current factual information, or an understanding of time-related relationships. Instead of solving this problem with individual hand-designed systems for every tool that could be used, Toolformer looks to determine whether an LLM can self-learn when a specific tool might be helpful, which tool will be the best to use in a specific situation, and how to implement the usage of this tool, with minimal human supervision.

Methodology. The approach to solving this problem is fairly simple. The researchers began with a small set of hand-written samples for several APIs, including a calculator, a Wikipedia search engine, a questions-answering (QA) system, a translator, and a calendar, and from here, the model creates a larger training set. This is done by sampling the possible locations an API call could be inserted, as well as the arguments passed to such a call, via in-context prompting. Then these candidate calls are executed, and only the ones that improve the prediction of the following tokens by a set threshold are kept. From here, the remaining examples create a better API-annotated dataset, which was used to train their LLM. The key difference from previous methods was the loss-based filtering step. Instead of asking humans to label if a tool call was useful, Toolformer uses its own model confusion to predict what comes next, avoiding the need for slower, manual annotations while still keeping successful tool calls.

Results. Toolformer had strong zero-shot learning improvements on several tasks, such as ASDiv, SVAMP, and MAWPS, factual completion tasks from LAMA, and temporal reasoning on DATESET. One of the biggest results was that the 6.7 Toolformer model outperformed GPT-3 on several arithmetic benchmark. This experiment proved that a smaller model with the right tool can perform better than a larger-scale general model without the tool.

Strengths. The self-supervised training pipeline is the biggest strength of Toolformer. Essentially, the model learns to use tools without task-specific labels, which means they are usable and scalable across other general domains and APIs. The aforementioned filtering mechanism is also a big benefit of Toolformer. The paper conducted a scaling ex-

periment, which showed that tool use for text generation only provided performance improvement over memory-only text generation above 775M parameters, demonstrating the computational benefits of filtering for tool calls at larger scales.

Weaknesses. Toolformer still has several limitations. First, the model could only call one API per input at inference time. This avoids looping when calling tools, but also means that multi-tool calls were not tested by the model, as they might be needed for more complex tasks. More advanced CAD requests, with multiple inputs, would not be achievable with the system’s current capability. Additionally, candidate API calls were sampled independently for each tool, so the output of one tool was never used as an input to another tool. Therefore, linked tool usage was not tested. Finally, the tested calculator tool was very limited, supporting only 4 mathematical operations. This was enough to create the benchmarks within the paper, but it is not on the scale needed for advanced scientific/engineering work, where significantly more operations would be necessary to carry out more complex operations.

2.2 DECINT: Natural Language Decomposition and Interpretation of Complex Utterances

Introduction and Motivation. DECINT (Jhamtani et al., 2023) looks to solve a different, but complimentary problem, relating to how complex user-inputs inputted to NL-to-program interfaces get broken down. The main idea is that LLMs, as they are trained on massive text datasets, learn general planning abilities and are able to recognize how complex inputs should be broken up into simpler phrases. However, traditional semantic parsers are only trained on simple, single-step commands. DECINT addresses this gap in knowledge by adding a step-by-step decomposition stage, which breaks the complex request into smaller sub-tasks before generating the final output program.

Methodology. The DECINT system works as a repeated step-by-step process. When given a complex request, it first puts together one natural-language sub-step based on the original request and on all of the earlier steps it has already produced. Then, this sub-step is turned into a small section of code with a separate NL-to-program model. This process keeps repeating until the full task has been broken down completely and converted

into code. This is different from normal chain-of-thought (CoT) prompting because DECINT combines the decomposition and code generation steps at the same time, instead of just planning the overall process via text and then generating code only at the end of the process. This is an important decision as it reduces confusion and drift from the input to the generated output as tasks become more and more complex. CoT outputs may create natural language steps and outputs that sound reasonable, but may not match what the API expects, because of the passing down of data from sub-step to sub-step. DECINT avoids this problem by influencing each future step with the code from earlier steps, meaning it checks the generated code rather than a textual final plan for each sub-step, improving the consistency of the overall program.

Results. On the DeCU benchmark, which looks at generating calendar and email management tasks, DECINT achieved 41% overall correctness, which was greater than DIRECT-PRED (direct creation of tasks), and CoT prompting, which scored 34% and 25%, respectively. Similar correctness scores were achieved for all architectures when using multiple LLMs, including GPT-4 and LLaMa-2-70B. Additionally, the use of ablation studies, where parts of a system were removed to see the importance of the part, showed that the decomposition and elementary utterance retrieval steps in the DECINT framework greatly improved its performance.

Strengths. The data efficiency of the DECINT method is one of its biggest strengths. The paper only utilized 10 complex training examples, but was still able to generalize well to new inputs with different structures. Obviously, such autonomy is necessary in practice as collecting labeled data for every complex NL-to-program problems would take lots of time, and is not feasible for every situation, while datasets for simpler commands are usually readily available. The other major strength of this method is the combination of decomposition and parsing. As each new step depends on the small generated code from the previous step, DECINT gets around a standard problem for CoT methods, where the final natural-language reasoning seems logical, but does not match the requirements of the target API. DECINT avoids this issue because each new step has full visibility into the code generated by all previous steps, keeping later reasoning anchored to the actual current program state instead of the overall target text description.

Weaknesses. DECINT has a few limitations that come with its strengths. The biggest issue is that outputs of code written from previous sub-steps are not used in future sub-steps. This means that programs that are designed to have multiple paths depending on the outcomes of earlier steps are not handled, such as two outcomes depending on whether a list to be checked is empty or not. Therefore, DECINT can only be used to solve inputs where a full plan is outlined ahead of time, which excludes many real-life tasks. Additionally, the benchmark used for evaluation is very limited. DeCU only holds about 200 complex utterances, all related to calendar and email fields. Using the DECINT architecture on a different topic would need new elementary utterances and complex examples, which may be time-consuming/expensive to create. 54% of predictions were not sufficient according to the WellForm metric, meaning code generated by the model is not always directly constrained to the target library. Finally, the best results for DECINT were created using proprietary models such as text-davinci-003 and GPT-4, meaning access is difficult, which is another weakness arising for successful DECINT implementation.

2.3 Gorilla: Large Language Model Connected with Massive APIs

Introduction and Motivation. Gorilla (Patil et al., 2024) looks to solve some of the issues with LLM-generated API calls at scale. Unlike Toolformer, which chooses from a predefined set of simple tools, or DECINT, which operates within a fixed domain (email/calendar management) with a predefined set of tasks, the goal of Gorilla is to select the correct API from over 1,600 frequently updated and overlapping possibilities from HuggingFace, TorchHub, and TensorFlowHub.

The main claim from this paper is that the standard for prompting and retrieval-augmented generation is insufficient because models sometimes hallucinate API arguments, mix up similar models, and don't respect user-defined constraints. Gorilla uses fine-tuning as the solution with a new training strategy, Retriever-Aware Training (RAT), that teaches the model to assign an appropriate weight for retrieved documentation at inference time.

Methodology. At a high level, Gorilla is built to generate the correct API call from a user instruction. To do so, the authors built APIBench, which is a dataset consisting of API documentation

from three model hubs, which was then turned into 10 example instructions and answer pairs for each model using GPT-4. The Gorilla model itself is a fine-tuned LLaMA-7B model, trained on these instruction pairs. In RAT, retrieved documentation is added to the training prompt even if that documentation is not fully correct. This teaches the model not to just copy from retrieved text, but to decide how much of the retrieved information should be trusted when creating the final API call. The evaluation metric is an Abstract Syntax Tree, which is a subtree matching metric. It verifies the structure of the generated API call against the structure of a reference call. Evaluation is focused on the functionality of the API call instead of the wording of the call. This is more substantial than something like an exact string match, as two API calls can look different in text while achieving the same output.

Results. Gorilla was found to perform better than GPT-4, GPT-3.5, and Claude on zero-shot API call generation, especially on TensorFlow Hub, where it reached 83.79% as compared to 18.20% for GPT-4. When Gorilla is able to look up relevant API documentation first, it achieves better performance across all three model hubs and nearly brings hallucination rates to zero. The paper demonstrated that RAT helps the model stay effective even when API documentation changes at test time, which is very practical as libraries and model hubs are typically updated often.

Strengths. RAT is by far the biggest strength of the Gorilla paper. By training the model with sometimes imperfect, retrieved data, the system treats retrieval as helpful evidence rather than a manual to be followed perfectly for results. This makes the model much more realistic and robust as compared to a system trained on only perfect retrieval. This is why Gorilla is one of the first models to show reliable API generation even as documentation changes over time. The AST subtree matching metric is another important takeaway from this paper. Unlike comparing the generated API call to a reference one, this metric can recognize when two different calls achieve the same task, and can assign partial credit when a function choice is correct, but a few input arguments are incorrect. The strong correlation between AST accuracy and human evaluation presented in this paper shows the usefulness of the metric.

Weaknesses. Unfortunately, similarly to Toolformer, Gorilla was purposefully kept limited in scope because it generates only a single API call for each instruction. This model works well for simple API selection tasks, but it is not set up for success with more complex workflows that require multiple steps. The tested model cannot put together several API calls or branch based on previous results, which is a limitation acknowledged by the authors, limiting feasibility. The training data is also limited as the library of APIs solely consists of machine learning APIs from HuggingFace, TorchHub, and TensorFlowHub. It was not explicitly tested whether this fine-tuning approach would work for other types of APIs. Additionally, the performance of RAT seemed to directly depend on the quality of retrieval. When the retrieved documentation significantly differed from the true API, performance dropped sharply, even sometimes below the no-retrieval baseline value.

2.4 Comparative Analysis

The three papers study similar general problems, but each one focuses on a different part of the challenge. Toolformer mainly focuses on self-supervised tool learning, showing that a model can learn when and how to use tools without large labeled datasets, but it is limited when tasks require multiple connected steps. DECINT focuses more on compositionality and few-shot generalization by breaking complex requests into smaller sub-tasks, but it still cannot use execution results from earlier steps to guide later ones. Gorilla focuses on accurate API selection and reducing hallucinated calls through fine-tuning and retrieval-aware training, but it is mainly limited to generating a single API call at a time.

A natural next step, and the one CADFormer attempts, is to combine these ideas. In particular, CADFormer uses a DECINT-style decomposition process to break a complex CAD request into smaller modeling operations, while also using Gorilla-style retrieval-augmented generation to ground each step in the relevant API documentation. CAD makes this a particularly demanding test case. Unlike calendar tasks in DeCU or model lookups in APIBench, CAD generation requires spatial reasoning, numerical accuracy, and strong compositional consistency, since the geometric result of one step directly affects the correctness of the next.

3 CADFormer: Implementation

3.1 System Overview

CADFormer is designed as a five-step framework that takes an input complex natural-language CAD request and turns it into an executable CadQuery Python program. The stages are:

1. **Preprocessing & Normalization:** The complex natural-language CAD request is preprocessed by normalizing units and resolving any missing or implied dimensions.
2. **DECINT-Style Decomposition:** As demonstrated in the DECINT paper, the input request is broken down into a series of smaller, natural-language modeling steps.
3. **Retrieval-Augmented Code Generation:** For each of these smaller steps, the most relevant CadQuery API documentation is retrieved and used to create a Python code block, for the step.
4. **Program Assembly & Execution:** Once all steps have their Python code generated, the code pieces are assembled into a full script to be executed by CadQuery.
5. **Feedback Collection:** Finally, the system checks whether execution was successful or not, and provides warnings/errors at the stop level anytime something goes wrong.

3.2 Elementary Operation Library

Similar to the fixed domain library used in DECINT, CADFormer relies on a set of 34 elementary CadQuery operations that define the basic actions available to the system. Each operation is paired with a sample natural-language description, a formal definition of how to call the function, allowable constraints for the operation, and an example of how it is used. These operations are grouped into five categories, including primitive creation, boolean operations, transforms, feature operations, and pattern operations. The library is stored as a JSON file and serves two main purposes for the rest of the implementation. It acts as the retrieval database used during the BM25 search (utilized retrieval algorithm) and also provides the set of valid operations for reference during evaluation. Both the decomposition and code generation stages use GPT-4 via the Azure OpenAI API as the underlying language model.

3.3 Decomposition Stage

The decomposition model is prompted with eight example decompositions of complex CAD requests and then asked to generate one step at a time based on the original request, the previous natural-language steps, and the earlier code fragments. In the CAD domain, this process is much more difficult than in DECINT’s implementation because the model must keep track of specific geometric objects and the spatial relationships between them, instead of being limited to calendar/email tasks. To help with the tracking of these relationships, later steps need to refer to results from earlier steps, such as “the cylinder from step 2” or “the top face of the box from step 1.” To make this easier to reference during code generation, the system uses explicitly named references from each intermediate step, labeled as s1, s2, etc. Finally, decomposition stops when the model either outputs a special <DONE> token or when a maximum of 12 steps is reached.

3.4 Retrieval-Augmented Code Generation

For every split, natural-language step, CADFormer retrieves the top three most relevant elementary operations from the operation library using a BM25 search. The retrieved documentation, which includes the operation name, signature, description, and example, is then added to the code generation prompt in a retrieval style similar to the one used by Gorilla. The code generation model then produces a CadQuery Python fragment for the current step using the original request, all calculated earlier steps, and subsequent earlier code fragments, and the retrieved documentation. A key improvement, specific to testing a CAD model, is the addition of a constantly updating context summary, which lists the variables created so far and their inferred geometric types, such as solid, face, edge, or wire. This helps reduce a common failure mode in NL-to-CAD generation, where the model might apply an operation to the wrong type of geometric object.

4 Evaluation

4.1 Benchmark Design

To be able to evaluate CADFormer, a benchmark of 36 natural CAD prompts with three types of complexity levels was created. The first level, simple inputs, contains 12 prompts, which were just simple command strings that focus on a single action from CadQuery, such as "Create a Cylinder with 10mm radius and 25mm height". The second

level consists of 12 more prompts, and includes multi-step Boolean and transformation operations, such as "Create a hollow rectangular tube with 2mm wall thickness and a chamfered inlet". The last level, consisting of the final 12 prompts, was created as an even more complex version of the prompts from the second level. The prompts contained between 6 and 10 grouped commands nested within the overall input, relating to more realistic patterns, conditional geometry, and part constraints that may be seen in actual CAD generation. All the prompts were put together and cross-checked with CadQuery implementation limitations to make sure the end result was feasible.

4.2 Metrics

The evaluation of the system is done with three calculated metrics. Execution Success Rate (ESR) measures the percentage of prompts for which the final assembled CadQuery program is able to run without error. Note, this does not guarantee that the CadQuery is what was requested from the input; rather, it checks that the generated program is actually executable. Step Accuracy (SA) measures the percentage of intermediate sub-steps that are formed correctly, meaning they use valid syntax and only reference defined variables and operations from the library. I compare four settings: DIRECT, which generates CadQuery code directly from the full prompt; Chain of Thought (CoT), which first generates a full natural-language plan before code; CADFormer-NoRetr, which does the decomposition process without any retrieval; and CADFormer, which is the full system (with retrieval).

4.3 Results

Results can be seen in Table 1 below. Overall, CADFormer has the highest ESR across all test prompts regardless of complexity, at 86.1%, which was higher than all three baselines. There were a few significant takeaways from the results of Table 1. First, the gap between CADFormer-NoRetr (52.8%) and CADFormer (86.1%) shows the specific impact that retrieving tangible API documentation at the time of code generation decreases step-level failures by a non-trivial amount. This is similar to the main finding of Gorilla, where the implementation of API retrieval before API calls reduces hallucination by the LLM. For our case, the retrieval documentation helped prevent generating potential false CadQuery functions or invalid CadQuery input arguments. Second, CoT

(69.4%) performed worse than DIRECT (77.8%), even though CoT created a textual plan before generating CadQuery code. This suggests that generating a complete natural-language plan before generating code may introduce additional failure points without providing meaningful grounding benefits, at least in the CAD domain. This finding, in our specific CAD generation setting, contrasts with that of DECINT’s argument for interleaving. Generating a complete NL plan before putting together code performed worse than direct generation, which might be because GPT-4 can partially ground future steps without explicit program context, or because CoT introduces more opportunities for output formatting errors across two separate generation phases. Third, for L3 ESR, the value unexpectedly increased from L2 ESR for three out of the four methods. This is most likely due to the small sample size tested — only 12 prompts each for L2 and L3. At this scale, a single additional failure represents an 8.33 percentage point drop per tier, meaning tier-level ordering is sensitive to individual prompt difficulty rather than reflecting a reliable trend across difficulty levels. A larger benchmark with more prompts per tier would be required to draw statistically robust conclusions about difficulty scaling.

4.4 Comparison to Paper Results

DECINT reported 41% overall correctness on the DeCU benchmark. For DECINT, the relative benefit of using interleaved decomposition vs direct prediction was 7%. Likewise, in our setting, CADFormer improved over the direct code generation (DIRECT) by approximately 8% on ESR, which was setup as a similar metric to the one used in DECINT. Interestingly, CADFormer-NoRetr, which used the same interleaved decomposition of the input as the ordinary CADFormer architecture did, showed a significant drop in performance compared to DIRECT (52.8% vs 77.8% ESR), indicating that in the CAD generation landscape, natural language decomposition without API grounding can actually hurt performance by introducing more steps where errors can accumulate.

Gorilla’s decrease in LLM hallucination due to retrieval is analogous to the ESR gap between CADFormer (86.1%), and CADFormer-NoRetr (52.8%). In Gorilla, the retrieval process improves accuracy by between 12-23% depending on which hub was used. The observed 33.3% ESR improvement with the usage of retrieval exceeds that range,

Method	ESR	L1	L2	L3	SA
DIRECT	77.8	91.7	75.0	66.7	—
COT	69.4	91.7	75.0	75.0	—
CADformer-NoRetr	52.8	75.0	41.7	41.7	100.0
CADformer	86.1	100.0	83.3	75.0	100.0

Table 1: Evaluation results on CADBench

suggesting that retrieval grounding is even more critical in the CAD domain than in ML API selection, likely because CadQuery’s selector syntax and operation chaining are harder for a model to recall correctly from memory than standard ML API names.

4.5 Brief Failure Analysis

When testing for step-level failures, several types of failures were discovered that are relevant to the CAD domain. The most frequent type is selector syntax that is invalid, meaning that the model creates a selector string that seems valid but is invalid according to the rules of the CadQuery selector grammar. For instance, it might create selectors of the form "`|Z` and `>Z'`" instead of selectors that follow a valid pattern like "`faces(>Z').edges()`". This issue is similar to the hallucination phenomenon described in the Gorilla paper, where the model creates its own selector that does not really exist in the API documentation. Adding a BM25 retrieval system helps reduce this problem, although it has not solved it because the retrieved API documents contain mainly information about operation signatures and examples.

Another failure mode that arose during this implementation is the violation of the order of operations. Some of the steps produced through this decomposition technique could be geometrically correct themselves, but are placed in the wrong sequence. For example, the model may try to fillet an edge before the boolean cut that actually creates that edge has been performed. This type of failure is far more common in CAD than in the domains used in DeCU and APIBench, since ordering does not play any role in doing calendar operations or performing machine learning API requests.

The third type of failure occurs mostly in the DIRECT and CoT baselines and involves output inconsistencies with respect to output formatting. At times, the model returns code enclosed in markdown fences such as "`python`". The inclusion of these fences will automatically lead to syntactic errors when the generated output is run as a script.

This is mainly an issue with the pre-written prompts that show up in whole-program generation. Conversely, step-by-step generation is naturally more stable, because each step is composed of one line of code at most.

By combining the above types of failures, it can be concluded that synthesis of the CAD program may need to consider using a constrained breakdown based on CadQuery type grammar, tighter topological ordering constraints in the decomposition prompt, and better control of the output format.

5 Limitations

Evaluation scale and noise. The benchmark dataset has only 36 prompts in three levels of difficulty, which may not provide a sufficient basis to make statistical findings on trends according to different difficulty levels due to the limited number of prompts used. Given this number, any change in the success rate or failure rate adds or subtracts from the percentage by about 8.33%. Increasing the number of prompts within the benchmark dataset would allow for stronger claims about performance trends.

Silent geometric errors. The ESR metric simply checks if there was any Python error when executing the generated code, but not the correctness of the final geometric result. It is possible in CadQuery that the code can generate results without generating any errors, but generate the wrong shape. For example, selecting the wrong face during selection, having too large a radius value for a fillet that is ignored by the compiler, or performing the wrong Boolean operation, resulting in wrong topology generation. These problems cannot be identified through execution and must be checked manually through geometric evaluation, where you compare the output STEP file with the desired one. This is a big challenge in evaluating generative CAD that does not appear in text benchmarks like within DeCu.

Step accuracy inflation. Step Accuracy, on the other hand, assesses only the syntax correctness of

the generated coding instruction. What that means is that the variables used in the process will be assigned correctly, and the names defined earlier will be used correctly too. However, the metric does not evaluate whether the CADQuery operation is being performed as it should or whether the numerical parameters used are correct. This means that a step can be accurate syntactically yet generate the wrong geometry.

LLM API dependency. CADformer depends on a proprietary LLM API, specifically Azure OpenAI GPT-4, for both decomposition and code generation. This creates cost, rate-limit, and reproducibility concerns. The reported results are also subject to API version changes and some degree of non-determinism across runs. In practice, rate limiting required inserting artificial delays between calls, which increased the total runtime of the benchmark substantially.

CadQuery API coverage. The CADformer relies on a proprietary LLM API, namely the Azure OpenAI GPT-4 API, for decomposition and code synthesis. Issues arise from costs, rate limitations, and reproducibility when not using this specific LLM. The findings are also sensitive to the evolution of the API itself and possess a certain level of non-determinism in their execution. During testing, rate limiting was required to not overload the LLM, adding a pause between API calls, thus expanding the benchmark time considerably.

6 Conclusion

The review of the three articles focuses on three important aspects of using LLMs for tool usage and API-based program synthesis. The first aspect covered by the paper Toolformer is that an LLM is capable of acquiring skills of tool usage in a mostly self-supervised way; however, the approach is limited in that it works only in terms of one-step tool invocation. The second paper, DECINT, demonstrates that hierarchical decomposition can help improve compositional generalization in dealing with complex requests; however, this model is not capable of utilizing the results of previous executions to inform further steps. Gorilla demonstrates that retrieval-aware finetuning has nonzero effects on API selection and can greatly reduce hallucinated calls at scale, but is limited to single API calls at any time.

All these concepts come together through CAD-

former in the context of CAD geometry, proving that interleaved retrieval and decomposition is viable in a more challenging task space. As our experiments have shown, CADformer achieves an 86.1% execution success rate, outperforming all three baselines. The 33.3 percentage point performance gap between CADformer and CADformer-NoRetr directly validates the key argument made by Gorilla on the significance of grounding generation in API documentation. Yet, the underperformance of CADformer-NoRetr (52.8%) compared to DIRECT (77.8%) also constitutes an interesting negative finding. This indicates that in a task such as CAD, in which both type constraints and spatial order play a pivotal role, decomposition alone falls short without proper API grounding and can actually introduce additional failure points compared to direct generation.

Further research would be best directed at addressing the failures observed through the tests conducted by CADformer. More specifically, using constrained decoding for CadQuery type grammar, adding topological ordering checks between decompositions, and fine-tuning with examples from CAD instructions-to-programs similar to what was done by Gorilla’s APIBench approach, may mitigate some of these failures.

References

- Harsh Jhamtani, Hao Fang, Patrick Xia, Eran Levy, Jacob Andreas, and Benjamin Van Durme. 2023. [Natural language decomposition and interpretation of complex utterances](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1–21.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. [Gorilla: Large language model connected with massive apis](#). In *Advances in Neural Information Processing Systems*, pages 1–22.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. [Toolformer: Language models can teach themselves to use tools](#). In *Advances in Neural Information Processing Systems*, pages 1–13. Curran Associates, Inc.